Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

Neptun kód:	**Y1FBMY**	Név: **Benedek Marcell**
Beadás verziószáma:	1.	

Feladat

Időjárás előrejelzés **

Település valamikor minimális hőmérséklettel

A meteorológiai intézet az ország N településére adott M napos időjárás előrejelzést, az adott településen az adott napra várt legmagasabb hőmérsékletet.

Készíts programot, amely megadja azokat a településeket, amelyeken előfordul valamelyik napi előrejelzések minimuma!

Bemenet

A *standard bemenet* első sorában a települések száma ($1 \le N \le 1000$) és a napok száma ($1 \le M \le 1000$) van. Az ezt követő N sorban az egyes napokra jósolt M hőmérséklet értéke található ($-50 \le H_{i,j} \le 50$).

Kimenet

A *standard kimenet* első sorába azon települések T számát kell kiírni, amelyeken előfordul valamelyik napi előrejelzések minimuma! Ezt kövesse ezen települések sorszáma, növekvő sorrendben!

Példa

Bemenet	Kimenet
3 5 10 15 12 10 10 11 11 11 11 20 12 16 16 16 20	2 1 2

Korlátok

Időlimit: 0.1 mp.

Memórialimit: 32 MB

Specifikáció

https://progalap.elte.hu/specifikacio/v1/?uuid=e5bfbaa8-7275-4691-9e9f-92535c894e4c

Be: n∈N, m∈N, h∈N[1..n,1..m]

Sa: mini∈N[1..m]

Ki: szam∈N, sorsz∈N[1..]

Fv: minsor: N ->N, minsor(i)=**MIN**(j=1..n, h[j,i]).minért

Fv: minik: N[,]->N[],

minik(h)=**MÁSOL**(i=1..m, minsor(i))

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

Fv: eleme: N -> L,

eleme(j)=**VAN**(k=1..n, mini[k]=h[j,k])

Ef: 1<=n<=1000 és 1<=m<=1000 és ∀i∈[1..n]:(∀j∈[1..m]:(-50<=h[i,j]<=50))

Uf: mini=minik(h) és (szam,sorsz)=**KIVÁLOGAT**(i=1..n, eleme(i), i)

Sablon

Másolás
sablon

i		y
e	→	f(e)
e+1	→	f(e+1)
...	→	...
u	→	f(u)

Feladat

Adott az egész számok egy [e..u] intervalluma és egy f:[e..u]→H függvény. Rendeljük az [e..u] intervallum minden értékéhez az f függvény hozzá tartozó értékét!

Specifikáció

```
Be: e∈Z, u∈Z
Ki: y∈H[1..u-e+1]
Ef: -
Uf: ∀i∈[e..u]:(y[i-e+1]=f(i))
```

Rövidítve:

```
Uf: y=MÁSOL(i=e..u, f(i))
```

Algoritmus

```
i=e..u
  y[i-e+1]:=f(i)
```

Változó
i:Egész

Eldöntés
sablon

Feladat

Adott az egész számok egy [e..u] intervalluma és egy T:[e..u]→Logikai feltétel. Határozzuk meg, hogy van-e az [e..u] intervallumnak olyan eleme, amely kielégíti a T feltételt!

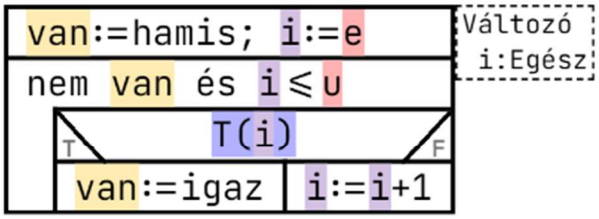
Specifikáció

Be: e∈Z, u∈Z
Ki: van∈L
Ef: -
Uf: van=∃i∈[e..u]:(T(i))

Rövidítve:

Uf: van=**VAN**(i=e..u,T(i))

Algoritmus



Kiválogatás
sablon

i		T(i)		f(i)
e	→	HAMIS		
e+1	→	IGAZ	→ 1	f(e+1)
e+2	→	IGAZ	→ 2	f(e+2)
u	→	HAMIS		

Feladat

Adott az egész számok egy [e..u] intervalluma, egy ezen értelmezett T:[e..u]→Logikai feltétel és egy f:[e..u]→H függvény. Határozzuk meg az f függvény az [e..u] intervallum azon értékeinél felvett értékeit, amelyekre a T feltétel teljesül!

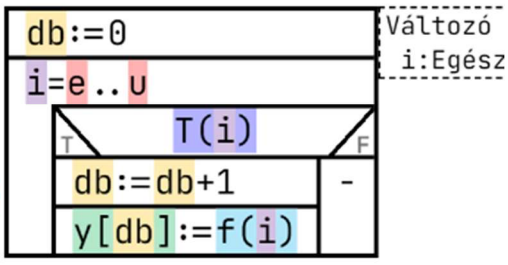
Specifikáció

Be: e∈Z, u∈Z
Ki: db∈N, y∈H[1..db]
Ef: -
Uf: db=**DARAB**(i=e..u,T(i)) és
∀i∈[1..db]:(
∃j∈[e..u]:T(j) és y[i]=f(j))
és y⊆(f(e),f(e+1),...,f(u))

Rövidítve:

Uf: (db,y)=**KIVÁLOGAT**(i=e..u,T(i),f(i))

Algoritmus



f és T között azáltal lesz kapcsolat, hogy a bemenet ugyanazon elemére hivatkoznak

Minimumkiválasztás
sablon

Feladat

Adott az egész számok egy [e..u] intervalluma és egy f:[e..u]→H függvény. A H halmaz elemein értelmezett egy teljes rendezési reláció. Határozzuk meg, hogy az f függvény hol veszi fel az [e..u] nem üres intervallumon a legkisebb értéket, és mondjuk meg, mekkora ez a minimális érték!

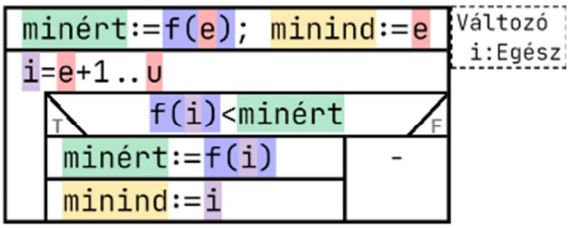
Specifikáció

Be: e∈Z, u∈Z
Ki: minind∈Z, minért∈H
Ef: e<=u
Uf: minind∈[e..u] és
∀i∈[e..u]:(f(minind)<=f(i)) és
minért=f(minind)

Rövidítve:

Uf: (minind,minért)=MIN(i=e..u,f(i))

Algoritmus



Visszavezetés

Minimumkiválasztás:

i=e..u ~ j=1..n

f(i) ~ h[j,i]

minind ~

minért ~ MIN(...).minért

MÁSOLÁS:

y ~ minik(h)

e..u ~ 1..n

f(i) ~ minsor(i)

Eldöntés:

van ~ eleme(j)

i=e..u ~ k=1..n

T(i) ~ mini[k]=h[j,k]

Kiválogatás:

db ~ szam

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

y ~ sorsz

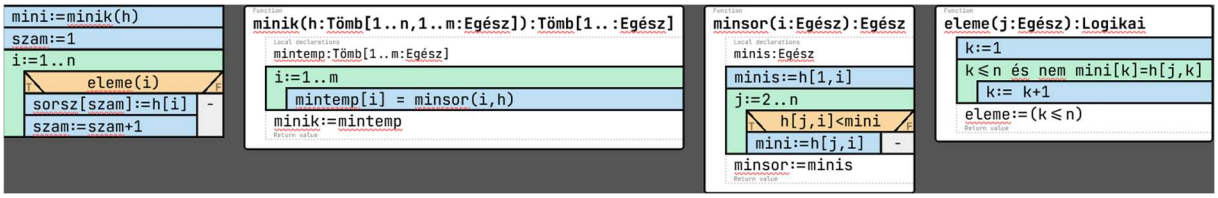
e..u ~ 1..n

T(i) ~ eleme(i)

f(i) ~ i

Algoritmus

https://progalap.elte.hu/stuki/v1/?uuid=2c3b83f1-fbb5-482e-80a2-7330eca126e0



Kód (alacsony szintű)

```
using System;
using System.Collections.Generic; // Listához kell


namespace beadxmod
{
  internal class Program
  {
    static int minsor(int colIndex, int[,] h, int n)
    {
      int mini = h[0, colIndex];
      for (int i = 1; i < n; i++)
      {
        if (mini > h[i, colIndex])
        {
          mini = h[i, colIndex];
        }
      }
      return mini;
    }
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

```
static int[] minik(int[,] h, int n, int m)
{
   int[] minik = new int[m];
   for (int j = 0; j < m; j++)
   {
      minik[j] = minsor(j, h, n);
   }
   return minik;
}

static bool eleme(int[] minik, int[,] h, int rowIndex, int m)
{
   int k = 0;
   while (k < m && minik[k] != h[rowIndex, k])
   {
      k++;
   }
   return (k < m);
}

static void Main(string[] args)
{
   string sor = Console.ReadLine();
   if (sor == null) return;

   string[] db = sor.Split(' ');
   int n = int.Parse(db[0]);
   int m = int.Parse(db[1]);

   int[,] h = new int[n, m];
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

Kód (magas szintű)

```
 for (int i = 0; i < n; i++)
        {
           string[] soradatok = Console.ReadLine().Split(' ');
           for (int j = 0; j < m; j++)
           {
              h[i, j] = int.Parse(soradatok[j]);
           }
        }

        int[] mini = minik(h, n, m);

        List<int> talalatok = new List<int>();

        for (int i = 0; i < n; i++)
        {
           if (eleme(mini, h, i, m))
           {
              talalatok.Add(i + 1);
           }
        }

        Console.Write($"{talalatok.Count} ");

        Console.WriteLine(string.Join(" ", talalatok));
     }
  }
}
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

```
using System;
using System.Collections.Generic;
using System.Linq;
using magas_szintű_mintamegvalósítások;

namespace beadxmod_high_level
{
  internal class Program
  {
    static void Main(string[] args)
    {
      int[,] matrix = Beolvasas();

      if (matrix == null) return;

      int[] eredmeny = Feldolgozas(matrix);

      Kiiras(eredmeny);
    }

    static void Uzenet(string szoveg)
    {
      if (!Console.IsInputRedirected)
      {
        Console.Write(szoveg);
      }
    }

    static int[,] Beolvasas()
    {
      Uzenet("Kérem az N (sorok) és M (oszlopok) számát: ");
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

```
        string sor = Console.ReadLine();
        if (string.IsNullOrEmpty(sor)) return null;

        string[] db = sor.Split(new char[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);
        int n = int.Parse(db[0]);
        int m = int.Parse(db[1]);

        int[,] h = new int[n, m];

        Uzenet($"Kérem a {n} db sort, soronként {m} számmal:\n");
        for (int i = 0; i < n; i++)
        {
          Uzenet($"{i + 1}. sor: ");
          string[] soradatok = Console.ReadLine().Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
          for (int j = 0; j < m; j++)
          {
            h[i, j] = int.Parse(soradatok[j]);
          }
        }
        return h;
      }

      static int[] Feldolgozas(int[,] matrix)
      {
        int n = matrix.GetLength(0);
        int m = matrix.GetLength(1);

        int[] minik = Mintak.Masol(0, m - 1, j =>
          Mintak.Min(0, n - 1, i => matrix[i, j]).minert
        );
```

```
        int[] talalatok = Mintak.Kivalogat(0, n - 1,
            i => Mintak.Van(0, m - 1, j => matrix[i, j] == minik[j]),
            i => i + 1
        );

        return talalatok;
    }

    static void Kiiras(int[] eredmeny)
    {
        Uzenet("Eredmény (darabszám és sorszámok): ");
        Console.Write($"{eredmeny.Length} ");
        Console.WriteLine(string.Join(" ", eredmeny));
    }
  }
}

namespace magas_szintű_mintamegvalósítások
{
  public static class Mintak
  {
    public static G Szumma<H, G>(int e, int u, Func<int, H> f, G kezd, Func<G, H, int, G> add)
    {
      G s = kezd;
      for (int i = e; i <= u; i++)
      {
        s = add(s, f(i), i);
      }
      return s;
    }
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

```
public static H[] Masol<H>(int e, int u, Func<int, H> f)
{
  return Szumma(e, u, f, new List<H>(), (s, p, i) =>
  {
    s.Add(p);
    return s;
  }).ToArray();
}

public static H[] Kivalogat<H>(int e, int u, Func<int, bool> t, Func<int, H> f)
{
  return Szumma(e, u, f, new List<H>(), (s, p, i) =>
  {
    if (t(i))
    {
      s.Add(p);
    }
    return s;
  }).ToArray();
}

public static (int minind, H minert) Min<H>(int e, int u, Func<int, H> f, Func<H, H, bool> isSmaller = null)
{
  bool _isSmaller(H a, H b)
  {
    if (a is IComparable<H> aa) return aa.CompareTo(b) < 0;
    else return isSmaller != null ? isSmaller(a, b) : false;
  }
  H minert = f(e);
```

```
        int minind = e;
        for (int i = e + 1; i <= u; i++)
        {
            if (_isSmaller(f(i), minert))
            {
                minert = f(i);
                minind = i;
            }
        }
        return (minind, minert);
    }

    public static (bool van, int ind) Keres(int e, int u, Func<int, bool> t)
    {
        int ind = e;
        while (ind <= u && !t(ind))
        {
            ind++;
        }
        bool van = ind <= u;
        return (van, ind);
    }

    public static bool Van(int e, int u, Func<int, bool> t)
    {
        return Keres(e, u, t).van;
    }
  }
}
```

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

Saját tesztesetek:

Teszteset 1:

3 5

10 15 12 10 10

11 11 11 11 20

12 16 16 16 20

Teszteset 2:

2 4

13 21 14 15

11 34 15 16

Teszteset 3:

4 4

5 6 7 8

1 2 3 4

0 2 3 4

3 3 4 5

Bíró pontszámok:

Total points: 100/

Test#	Point	...Verdict...	CPU time
1.1	4/4	Helyes	0.028 sec
2.1	4/4	Helyes	0.030 sec
3.1	4/4	Helyes	0.038 sec
4.1	4/4	Helyes	0.037 sec
5.1	4/4	Helyes	0.037 sec
6.1	5/5	Helyes	0.037 sec
7.1	5/5	Helyes	0.036 sec
8.1	5/5	Helyes	0.037 sec
9.1	5/5	Helyes	0.036 sec
10.1	5/5	Helyes	0.036 sec
11.1	5/5	Helyes	0.039 sec
12.1	5/5	Helyes	0.037 sec
13.1	5/5	Helyes	0.038 sec
14.1	5/5	Helyes	0.047 sec
15.1	5/5	Helyes	0.041 sec
16.1	6/6	Helyes	0.073 sec
17.1	6/6	Helyes	0.074 sec
18.1	6/6	Helyes	0.072 sec
19.1	6/6	Helyes	0.134 sec
20.1	6/6	Helyes	0.350 sec

Date of submission: 2026-01-11 18:25:35.0

Megoldás sablon „C” beadandó 1. fázishoz (lecserélendő a konkrét beadandó megnevezésére)

Total points: 100/

Test#	Point	...Verdict...	CPU time
1.1	4/4	Helyes	0.033 sec
2.1	4/4	Helyes	0.032 sec
3.1	4/4	Helyes	0.032 sec
4.1	4/4	Helyes	0.038 sec
5.1	4/4	Helyes	0.028 sec
6.1	5/5	Helyes	0.039 sec
7.1	5/5	Helyes	0.037 sec
8.1	5/5	Helyes	0.032 sec
9.1	5/5	Helyes	0.038 sec
10.1	5/5	Helyes	0.030 sec
11.1	5/5	Helyes	0.038 sec
12.1	5/5	Helyes	0.040 sec
13.1	5/5	Helyes	0.040 sec
14.1	5/5	Helyes	0.043 sec
15.1	5/5	Helyes	0.032 sec
16.1	6/6	Helyes	0.064 sec
17.1	6/6	Helyes	0.066 sec
18.1	6/6	Helyes	0.058 sec
19.1	6/6	Helyes	0.119 sec
20.1	6/6	Helyes	0.314 sec

Date of submission: 2026-01-11 21:08:48.0